

ผู้จัดทำ

66010106	นายจตุภัทร ขจรชัยกุล	Train and Test model, Pure Python
66010413	นางสาวนัทธมน บุญไให้	Database, Docker On Cloud
66010471	นางสาวปวีศา แก้วพันมา	หาข้อมูล, ทำรายงาน
66010827	นายศุภวิช ธีรัฐธรรม	All in One
66011390	นายธิตี ศรีประสงค์	Dataset, remote 3D printer

Spaghetti Shape Detection

การพิมพ์สามมิติ (3D Printing) เป็นเทคโนโลยีที่ได้รับความนิยมอย่างแพร่หลายในการสร้างต้นแบบและชิ้นงานจริง อย่างไรก็ตาม ความล้มเหลวระหว่างการพิมพ์ เช่น การเกิดเส้นสปาเกตตี (Spaghetti Failure) ยังคงเป็นปัญหาที่พบบ่อย และส่งผลให้สิ้นเปลืองเวลา วัสดุ และพลังงาน งานวิจัยนี้นำเสนอการพัฒนาโมเดลตรวจจับความล้มเหลวของการพิมพ์สามมิติแบบเรียลไทม์ โดยใช้สถาปัตยกรรม YOLOv8 (You Only Look Once Version 8) ซึ่งเป็นเทคนิคการตรวจจับวัตถุ (Object Detection) รุ่นล่าสุดจาก Ultralytics ที่มีความสามารถสูงในการประมวลผลภาพอย่างรวดเร็วและแม่นยำ ชุดข้อมูล (Dataset) ที่ใช้ในการฝึกและทดสอบโมเดลได้มาจากการเก็บภาพกระบวนการพิมพ์จากเครื่องพิมพ์ 3D เพียงเครื่องเดียว โดยใช้เส้นใย (filament) สีเดียวเพื่อจำแนกภาพออกเป็น 3 คลาส ได้แก่ normal print, printer head และ spaghetti failed ผลการทดลองแสดงให้เห็นว่าโมเดลสามารถตรวจจับความล้มเหลวได้ด้วยความแม่นยำเฉลี่ย (Accuracy) ประมาณ 90% ผลลัพธ์จากงานนี้สามารถต่อยอดสู่การพัฒนา ระบบตรวจจับความล้มเหลวอัตโนมัติสำหรับเครื่องพิมพ์ 3D ที่ทำงานได้อย่างมีประสิทธิภาพมากขึ้นในอนาคต โดยเฉพาะเมื่อมีการเพิ่มความหลากหลายของชุดข้อมูลและเงื่อนไขการพิมพ์ให้ครอบคลุมมากขึ้น

PROBLEM STATEMENT

การพิมพ์สามมิติ (3D Printing) แม้จะเป็นเทคโนโลยีที่ช่วยลดต้นทุนและเวลาในการผลิตต้นแบบ แต่ก็ยังคงประสบปัญหาความล้มเหลวระหว่างการพิมพ์ ซึ่งก่อให้เกิดการสูญเสียอย่างมีนัยสำคัญ โดยเฉพาะอย่างยิ่งในกรณีของงานพิมพ์ขนาดใหญ่ที่ใช้เวลาหลายชั่วโมง ความล้มเหลวเพียงครั้งเดียวสามารถทำให้สูญเสียวัสดุ (Filament) จำนวนมาก รวมถึงเวลาและพลังงานไปโดยเปล่าประโยชน์ การตรวจจับด้วยสายตามนุษย์เป็นสิ่งที่ทำได้ยาก และไม่สามารถเฝ้าสังเกตได้ตลอด 24 ชั่วโมง ส่งผลให้เมื่อเกิดความผิดพลาด ผู้ใช้มักพบปัญหาหลังจากงานพิมพ์เสียหายไปแล้ว (Circuit Digest, SimplyPrint)

อีกปัญหาสำคัญคือการขาดระบบติดตามและเฝ้าดูระยะไกล (Remote Monitoring) ผู้ใช้งานไม่สามารถตรวจสอบสถานะของเครื่องพิมพ์ได้เมื่ออยู่นอกสถานที่ เช่น ระหว่างทำงานหรือนอนหลับ ทำให้ไม่สามารถหยุดกระบวนการพิมพ์ที่ล้มเหลวได้อย่างทันท่วงที แม้จะมีโซลูชันอย่าง OctoPrint ที่ช่วยติดตามผ่านเครือข่ายได้ แต่ยังมีข้อจำกัดด้านการเข้าถึงจากภายนอก ซึ่งต้องอาศัยระบบคลาวด์หรือ AI เพิ่มเติมเพื่อให้สามารถควบคุมจากระยะไกลได้ (Obico, SimplyPrint)

นอกจากนี้ เครื่องพิมพ์ 3D ส่วนใหญ่ในท้องตลาด โดยเฉพาะรุ่นราคาประหยัด ยังขาดความสามารถในการตรวจจับความผิดพลาดอัตโนมัติในตัวเครื่อง ผู้ใช้ที่มีงบประมาณจำกัดจึงไม่สามารถเข้าถึงเทคโนโลยีเหล่านี้ได้ โครงการนี้จึงมุ่งเน้นการพัฒนา “ระบบตรวจจับความผิดพลาดอัจฉริยะ” ที่สามารถเชื่อมต่อและใช้งานร่วมกับเครื่องพิมพ์ 3D ทั่วไปได้อย่างอิสระ เพื่อเป็น “สะพานเชื่อม” ให้เครื่องพิมพ์รุ่นเก่าและยี่ห้อต่าง ๆ เข้าถึงฟังก์ชันตรวจสอบข้อผิดพลาดอัตโนมัติได้ โดยไม่จำเป็นต้องลงทุนซื้อเครื่องใหม่

DATA PREPARATION

Data Source

Positive image:

เก็บข้อมูลเอง ด้วยวิธีอัตโนมัติโอการทำงานของเครื่องพิมพ์ 3 มิติ และใช้ filament ของสาขา และค่อย ๆ แปลงแต่ละ frame เป็นไฟล์ภาพ

Negative image:

- animals-10. <https://www.kaggle.com/datasets/alessiocorrado99/animals10/data>
- daily object around the world:
<https://www.kaggle.com/datasets/humansintheloop/dollar-street-dataset>
- Pasta Classification Dataset
[:https://www.kaggle.com/datasets/aksha05/pasta-classification](https://www.kaggle.com/datasets/aksha05/pasta-classification)

Data Preprocessing

Roboflow Auto-Orient

Data Transformation

ใช้การ Augmentation ของ Roboflow โดยได้ผลลัพธ์เป็น 3 เท่า (1037 รูปภาพ) ของจำนวนภาพดั้งเดิม

- Horizontal Flip
- Grayscale 20% of the image
- Hue (สุ่มปรับช่วงตั้งแต่ -25% ถึง+25%)
- Saturation (สุ่มปรับช่วงตั้งแต่ -30% ถึง+30%)
- Brightness (Randomly adjust between -20% and +20%)

Module ที่ใช้ในการ Train โมเดล

Python	: 3.12.11	seaborn	: 0.13.2	scikit-learn:	1.6.1
Ultralytics	: 8.3.204	pandas	: 2.2.2	os	: builtin, no version
opencv	: 4.12.0	numpy	: 2.0.2	glob	: builtin, no version
matplotlib	: 3.10.0	torch	: 2.8.0+cu126	random	: builtin, no version

TRAIN MODEL

สถาปัตยกรรมที่ใช้

YOLOv8 (You Only Look Once Version 8) เป็นสถาปัตยกรรมการตรวจจับวัตถุ (Object Detection) จุดเด่นของ YOLOv8 คือความสามารถในการตรวจจับวัตถุได้อย่างรวดเร็วและแม่นยำภายในขั้นตอนเดียว (Single-Stage Detector) ทำให้เหมาะกับการใช้งานแบบเรียลไทม์ (Real-time Application)

YOLOv8 มีการออกแบบที่ทันสมัยกว่า YOLOv5–YOLOv7 โดยเพิ่มความยืดหยุ่นและประสิทธิภาพ เช่น

- Backbone ที่ปรับปรุงใหม่ → ใช้สถาปัตยกรรม CSP (Cross Stage Partial Network) และ C2f block ซึ่งช่วยให้การดึงคุณลักษณะของภาพ (Feature Extraction) มีประสิทธิภาพสูงขึ้น
- Neck และ Head ที่ยืดหยุ่น → ใช้ PAN-FPN (Path Aggregation Network + Feature Pyramid Network) เพื่อรวมคุณลักษณะจากหลายสเกล ทำให้ตรวจจับวัตถุได้ทั้งขนาดเล็กและใหญ่
- Anchor-free detection → YOLOv8 รองรับ anchor-free ทำให้ลดความซับซ้อนและสามารถเรียนรู้ bounding box ได้ยืดหยุ่นขึ้น
- รองรับงานหลายประเภท → นอกจาก Object Detection ยังรองรับ Image Classification, Instance Segmentation และ Pose Estimation
- ประสิทธิภาพสูง → ทำงานได้รวดเร็วบน GPU, CPU และรองรับการ deploy ไปยัง edge device เช่น Raspberry Pi, Jetson Nano, หรือแม้กระทั่ง Mobile

ความสามารถเหล่านี้ทำให้ YOLOv8 ได้รับความนิยมอย่างกว้างขวางในงานวิจัยและการประยุกต์ใช้งานจริง เช่น การตรวจจับความผิดปกติในการผลิต, การจราจรอัจฉริยะ, และการตรวจวัดทางการแพทย์ เป็นต้น

Hyper Parameter

- epochs = 100 รอบการเรียนรู้ ยิ่งมากยิ่งเรียนได้ลึก แต่เสี่ยง overfit ถ้า dataset เล็ก
- imgsz = 640 ขนาดภาพ input; ใหญ่ = แม่นยำขึ้นแต่ใช้เวลาและ VRAM มากขึ้น
- batch = 16 จำนวนภาพต่อรอบอัปเดต; ค่ามากช่วยให้ gradient เสถียร แต่กิน RAM สูง
- device = 0 บังคับใช้ GPU ตัวแรกในการฝึก
- amp = True เปิด mixed precision ฝึกเร็วขึ้นและประหยัด VRAM
- seed = 106 ตั้งค่าให้ผลการเทรนสามารถทำซ้ำได้ (reproducible)

โดยในการ Train model จะแบ่ง dataset เป็น 3 ส่วน ได้แก่

- Train set: 71%
- Validation set: 12%
- Test set: 17%

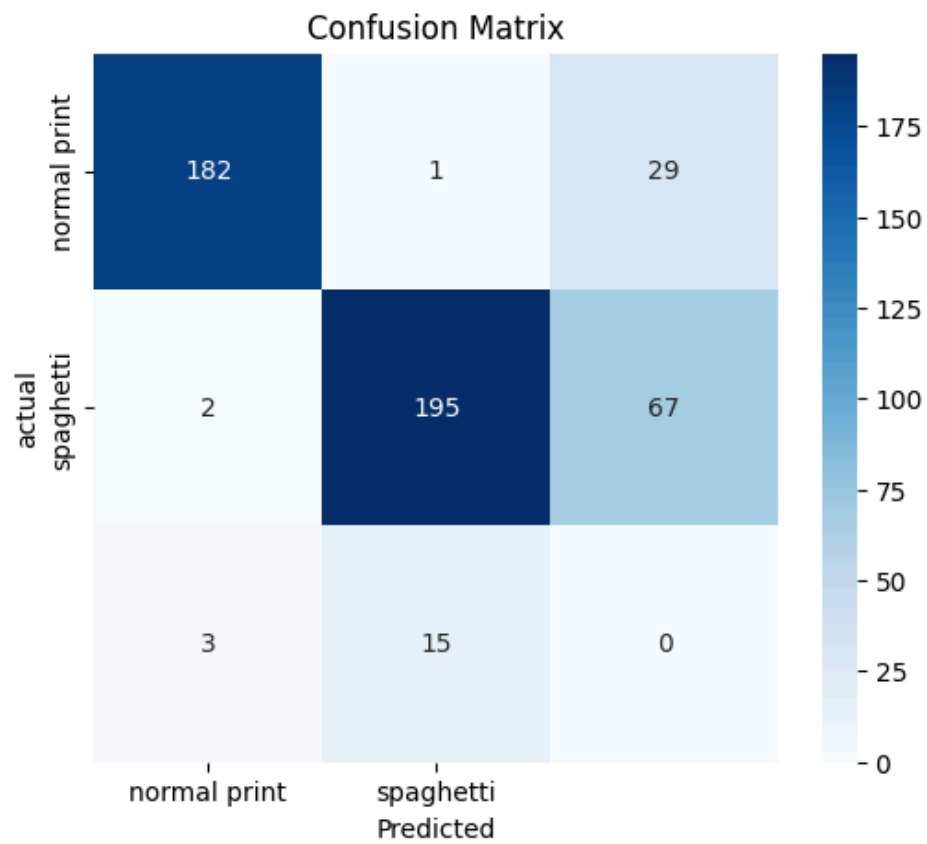
การประเมิน model

Evaluation

- **Precision** → บอกว่าสัดส่วนของที่โมเดลทำนายว่าเป็นคลาสนั้น ถูกต้องจริงกี่เปอร์เซ็นต์
- **Recall** → บอกว่าสัดส่วนของที่เป็นคลาสนั้นจริง ๆ โมเดลสามารถตรวจเจอได้กี่เปอร์เซ็นต์
- **F1-score** → ค่าเฉลี่ยเชิงฮาร์โมนิกระหว่าง Precision และ Recall เพื่อสมดุลทั้งสองด้าน
- **Support** → จำนวนตัวอย่างจริงของแต่ละคลาสที่อยู่ในชุดทดสอบ
- **Accuracy** → สัดส่วนของการทำนายที่ถูกต้องทั้งหมดเมื่อเทียบกับจำนวนตัวอย่างทั้งหมด
- **Macro avg** → ค่าเฉลี่ยแบบไม่ถ่วงน้ำหนักของ Precision, Recall และ F1-score ทุกคลาส
- **Weighted avg** → ค่าเฉลี่ยแบบถ่วงน้ำหนักตามจำนวนตัวอย่าง (Support) ของแต่ละคลาส

	Class	Precision	Recall	F1-score	Support
	normal print	0.9341	0.9103	0.9221	212
	spaghetti	0.9127	0.8720	0.8919	264
	accuracy			0.8891	476
	macro avg	0.9234	0.8912	0.9070	476
	weighted avg	0.9222	0.8891	0.9053	476

Confusion Matrix



การ เรียกใช้ MODEL

ในฝั่งเซิร์ฟเวอร์ (Backend) ของระบบ จะมีการสร้างอ็อบเจกต์ของ YOLOv8 ขึ้นมาจากโมเดลที่ผ่านการเทรนไว้ก่อนแล้ว โดยโหลดเพียงครั้งเดียวเมื่อเริ่มต้นระบบผ่านคำสั่ง `YOLO(model_path)` เพื่อความรวดเร็วในการเรียกใช้งาน ไม่ต้องโหลดซ้ำในทุกคำขอ จากนั้นจะมีฟังก์ชันหลัก

ชื่อว่า `detect` ที่รับหน้าที่ในการประมวลผลรูปภาพที่ผู้ใช้ทำการอัปโหลดเข้ามา โดยทันทีที่ได้รับไฟล์ภาพ ฟังก์ชันนี้จะทำการรันโมเดลผ่านคำสั่ง `model.predict` โดยตั้งค่า `threshold` ไว้เพื่อกรองค่าความมั่นใจของการตรวจจับ และจำกัดเฉพาะการตรวจจับวัตถุในคลาสที่ 0 และ 2 ซึ่งเป็นคลาสที่เกี่ยวข้องกับการตรวจสอบเส้นспаเกิดดีในชั้นงาน

หลังจากที่โมเดลทำการพยากรณ์ผลเรียบร้อยแล้ว ฟังก์ชันจะทำการวาดกรอบลงบนรูปภาพเพื่อแสดงตำแหน่งที่ตรวจพบ โดยหากเป็นบริเวณที่ตรวจพบว่าเป็นเส้นспаเกิดดี (ตามคำนิยามของคลาสที่กำหนด) จะวาดกรอบเป็นสีแดงเพื่อแสดงถึงตำแหน่งที่มีข้อผิดพลาดของชั้นงาน ส่วนบริเวณอื่นที่ระบบตรวจสอบว่าไม่มีปัญหา จะวาดกรอบเป็นสีเขียวแทนเพื่อแสดงว่าเป็นส่วนที่ปกติ จากนั้นระบบจะเลือกค่าความมั่นใจ (confidence score) ที่สูงที่สุดของการตรวจจับทั้งหมดมาใช้เป็นคะแนนหลักของภาพนั้น ๆ เพื่อใช้ในการตัดสินผลสุดท้าย ระบบจะประเมินสถานะของชั้นงานจากผลลัพธ์ทั้งหมด และสรุปเป็นสองสถานะหลัก ได้แก่ “Fail” หากพบวัตถุต้องห้ามในภาพ หรือ “Normal” หากภาพไม่มีข้อผิดพลาดใด ๆ ต่อจากนั้นระบบจะทำการบันทึกภาพที่ผ่านการติกรอบแล้วเป็นไฟล์ `.jpg` ลงในโฟลเดอร์ชั่วคราว (temp) ซึ่งเชื่อมโยงกับ session ของผู้ใช้นั้น โดยเฉพาะ เพื่อให้สามารถจัดการไฟล์ได้อย่างเป็นสัดส่วนไม่ปะปนกันระหว่างผู้ใช้ พร้อมกันนั้น ระบบยังจะบันทึกข้อมูลของผลลัพธ์ที่ได้ทั้งหมดลงในฐานข้อมูล (SQLite) ซึ่งจะประกอบไปด้วยข้อมูลสำคัญ เช่น ชื่อไฟล์ภาพ, เส้นทางของไฟล์ที่ผ่านการประมวลผล, คะแนนที่ทำนายได้, ชื่อโมเดลที่ใช้ และสถานะของชั้นงาน เพื่อให้สามารถเรียกดูย้อนหลังได้อย่างเป็นระบบ สุดท้ายเมื่อมีคำขอจากฝั่ง Frontend ผ่าน API ระบบจะเรียก

ฟังก์ชันที่ใช้ดึงข้อมูลจากฐานข้อมูลโดยอิงจาก card_id หรือ session_id แล้วจัดเตรียมผลลัพธ์เป็น JSON ซึ่งประกอบด้วย Path ของรูปภาพที่ตรวจสอบแล้ว พร้อมข้อมูลที่จำเป็น เช่น สถานะ, คะแนน, เวลาอัปเดต และโมเดลที่ใช้ เพื่อตอบกลับไปยังหน้าเว็บให้แสดงผลได้ทันที

ข้อจำกัดทางด้าน software และ/หรือ hardware วิธีแก้ปัญหา

1. ด้าน Hardware

ในการพัฒนาโมเดลตรวจจับความล้มเหลวของการพิมพ์สามมิติ ข้อมูลที่ใช้ในการฝึกถูกเก็บจาก เครื่องพิมพ์ 3 มิติเพียงเครื่องเดียว และใช้ เส้นใย (filament) สีเดียว ตลอดกระบวนการเก็บข้อมูล ส่งผลให้ข้อมูลมีความซ้ำซ้อนและขาดความหลากหลาย ทำให้โมเดลอาจไม่สามารถทำนาย ได้อย่างแม่นยำเมื่อพบภาพการพิมพ์ที่แตกต่างออกไป เช่น การใช้เส้นใยคนละสี หรือเครื่องพิมพ์คนละรุ่น

วิธีแก้ไข:

เพื่อเพิ่มความหลากหลายของข้อมูล ควรทำ Data Augmentation เช่น การปรับความสว่าง (Brightness), การกลับภาพ (Flip), และการปรับค่า Saturation หรือ Hue เพื่อให้โมเดลสามารถเรียนรู้จากข้อมูลจำลองที่ใกล้เคียงกับสถานการณ์จริงมากขึ้น

2. ด้าน Software

ข้อจำกัดหลักด้านซอฟต์แวร์คือการพึ่งพาไลบรารี Ultralytics YOLOv8 ซึ่งมีการอัปเดตบ่อย ทำให้บางฟังก์ชันหรือคำสั่งเปลี่ยนแปลงและอาจไม่เข้ากันกับเวอร์ชันเก่าของไลบรารีเสริม เช่น torch, opencv, หรือ numpy นอกจากนี้ การประมวลผลยังต้องใช้ทรัพยากรสูง โดยเฉพาะบน GPU ที่มีหน่วยความจำจำกัด เช่น NVIDIA T4 ซึ่งอาจเกิดการขัดข้องระหว่างการเทรน

วิธีแก้ไข:

ควรกำหนดเวอร์ชันของไลบรารีให้แน่นอนและแจ้งให้ชัดเจน เพื่อป้องกันปัญหาความไม่เข้ากัน (dependency conflict) และอัปเดตโค้ดให้สอดคล้องกับ YOLOv8 เวอร์ชันล่าสุดอยู่เสมอ

FUTURE WORK

- **การเพิ่มความหลากหลายของข้อมูล (Dataset Expansion):**

เก็บข้อมูลภาพจากเครื่องพิมพ์ 3 มิติหลายรุ่น และใช้เส้นใย (filament) ที่มีสีและวัสดุต่างชนิดกัน เพื่อให้โมเดลสามารถเรียนรู้ลักษณะความผิดพลาดได้หลากหลายยิ่งขึ้น รวมถึงขยายรูปแบบชิ้นงานให้ครอบคลุมลักษณะทางเรขาคณิตที่ซับซ้อนมากขึ้น

- **การเพิ่มมุมมองของข้อมูล (Multi-Angle Image Collection):**

ใช้ภาพจากมุมมองต่าง ๆ ในการเก็บภาพ เพื่อให้โมเดลสามารถเข้าใจรูปทรงของชิ้นงานในมิติที่หลากหลาย ลดข้อผิดพลาดจากการมองเห็นในมุมจำกัด และเพิ่มความแม่นยำในการตรวจจับ

- **การประยุกต์ใช้งานแบบเรียลไทม์ (Real-Time Implementation):**

พัฒนา Application ให้สามารถประมวลผลและตรวจจับข้อผิดพลาดได้แบบเรียลไทม์ เพื่อให้ระบบสามารถแจ้งเตือนหรือหยุดการพิมพ์โดยอัตโนมัติเมื่อเกิดความล้มเหลว เช่น การเกิดเส้นสปาเกตตี

- **การขยายประเภทของข้อผิดพลาดที่ตรวจจับได้ (Multi-Error Detection):**

เพิ่มการเรียนรู้ให้โมเดลสามารถตรวจจับข้อผิดพลาดอื่น ๆ ที่เกิดขึ้นระหว่างการพิมพ์ เช่น การแยกชั้น (Layer Separation) หรือ การหลุดจากฐานพิมพ์ (Bed Detachment) เพื่อให้ระบบมีความอัจฉริยะมากขึ้น